

Versione 1 dell'esercizio sui tipi di ML

Scrivere il tipo della funzione di ML `List.nth` che restituisce l'*i*-esimo elemento di una lista.

Versione 2 dell'esercizio sui tipi di ML

Scrivere il tipo della funzione di ML null che verifica se una lista è vuota.

Versione 3 dell'esercizio sui tipi di ML

Scrivere il tipo della funzione ML:

```
fun f (x, []) = 0  
  | f (x, y::z) = x=y orelse f (x, z);
```

segnalando eventuali errori.

Versione 4 dell'esercizio sui tipi di ML

Scrivere il tipo della funzione di ML `List.nth` che restituisce l'*i*-esimo elemento di una lista.

Versione 5 dell'esercizio sui tipi di ML

Scrivere il tipo della funzione di ML null che verifica se una lista è vuota.

Versione 6 dell'esercizio sui tipi di ML

Scrivere il tipo della funzione ML:

```
fn (x,L) => if L=[] then false else let val (y::R) = L in x=y end;
```

segnalando eventuali errori.

Versione 7 dell'esercizio sui tipi di ML

Scrivere il tipo della funzione di ML `hd` che restituisce la testa di una lista.

Versione 8 dell'esercizio sui tipi di ML

Scrivere il tipo della funzione ML:

```
fun f (x, []) = 0  
  | f (x, y::z) = x=y orelse f (x, z);
```

segnalando eventuali errori.

Versione 9 dell'esercizio sui tipi di ML

Scrivere il tipo della funzione ML:

```
fun f x [] = 0  
  | f x (y::z) = x=y orelse f x z;
```

segnalando eventuali errori.

Versione 10 dell'esercizio sui tipi di ML

Scrivere il tipo della funzione ML:

```
fun f x [] = false  
  | f x (y::z) = x=y orelse f x z;
```

segnalando eventuali errori.

Versione 11 dell'esercizio sui tipi di ML

Scrivere il tipo della funzione ML:

```
fn x => fn L => if L=[] then false else let val (y::R) = L in x=y end;
```

segnalando eventuali errori.